

NATURAL LANGUAGE PROCESSING

DTS305TC

HUAKANG LI

HUAKANG.LI@XJTLU.EDU.CN

SCHOOL OF AI AND ADVANCED COMPUTING



Xi'an Jiaotong-Liverpool University

西交利物浦大學



LAB 5 DOCUMENT CLASSIFICATION

week 3 day 1

<https://spotintelligence.com/2023/10/23/document-classification-python/>



OVERVIEW

- Basics of Document Classification
- Getting Started With Data Preparation
- Machine Learning Models for Document Classification



COLLECTING A LABELED DATASET

- The foundation of document classification is a labelled dataset. This dataset consists of text documents associated with a predefined category or label. The process of collecting such a dataset involves the following:
 - Data Sourcing: Identify and gather text documents covering the topics or categories you want to classify. This might involve web scraping, data extraction from databases, or using existing datasets.
 - Labelling: Manually or automatically assign labels to the documents. For instance, if you have a spam email dataset, you would mark each email as “spam” or “not spam.”
 - Balancing: Ensure that the dataset is balanced, meaning that each category has a sufficient number of documents for training. Imbalanced datasets can lead to biased models.



```
import pandas as pd
data = pd.read_csv('business_data.csv')
X = data['content']
y = data['category']
label_counts = y.value_counts()
print(label_counts)
```

```
category
tradee    1001
business   999
Name: count, dtype: int64
```



DATA CLEANING AND PREPROCESSING

- Before feeding the data into a classification model, cleaning and preprocessing it is essential. Data cleaning and preprocessing steps typically include the following:
 - Text Cleaning: Remove any noise from the text data, such as HTML tags, special characters, or non-textual elements.
 - Lowercasing: Convert all text to lowercase to ensure consistency in text representations. This prevents the model from treating “cat” and “Cat” as different words.
 - Tokenization: Split the text into words or subword units. Tokenization is crucial for representing text as numerical features.
 - Stop Word Removal: Eliminate common words like “and,” “the,” and “in” that carry little meaning for document classification tasks.
 - Stemming or Lemmatization: Reduce words to their root form to ensure that different inflexions of a word are treated as the same word (e.g., “running” and “ran” to “run”).



```
import nltk
from nltk.corpus import stopwords
from nltk.stem import
WordNetLemmatizer
from nltk.tokenize import
word_tokenize
import string
```

```
nltk.download('stopwords')
nltk.download('wordnet')
```

```
def preprocess_text(text):
    translator = str.maketrans("", "", string.punctuation)
    text_no_punct = text.translate(translator)
    text_lower = text_no_punct.lower()
    tokens = word_tokenize(text_lower)
    stop_words = set(stopwords.words('english'))
    filtered_tokens = [word for word in tokens if word not in
stop_words]
    lemmatizer = WordNetLemmatizer()
    lemmatized_tokens = [lemmatizer.lemmatize(word) for
word in filtered_tokens]
    preprocessed_text = ' '.join(lemmatized_tokens)
    return preprocessed_text
```



HOW CAN YOU HANDLE IMBALANCED DATASETS?

- Imbalanced datasets are standard in document classification, where some categories may have many more examples than others. To address this, consider techniques such as:
 - Oversampling: Create additional copies of minority class samples to balance the dataset.
 - Undersampling: Reduce the number of majority class samples to balance the dataset.
 - Synthetic Data Generation: Use techniques like SMOTE (Synthetic Minority Over-sampling Technique) to generate synthetic examples for minority classes.
 - Evaluation Metrics: Use appropriate metrics like F1-score, precision-recall, or ROC-AUC, which are more robust for imbalanced datasets than accuracy.



TEXT FEATURE EXTRACTION TECHNIQUES

- After data preprocessing, you need to convert the text data into numerical features that machine learning models can use.

Standard text feature extraction techniques include:

- Bag of Words (BoW): Represent each document as a vector of word frequencies. Each word is a feature, and its frequency in the document is its value.
- Term Frequency-Inverse Document Frequency (TF-IDF): Weigh words based on their importance in a document relative to their significance in a corpus. It helps identify words that are discriminative for a category.
- Word Embeddings: This method utilizes pre-trained word vectors (e.g., Word2Vec, GloVe, fastText) to represent words and documents in a continuous vector space.



```
# Vectorize the text data using TF-IDF
vectorizer = TfidfVectorizer(max_features=5000)
X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)
```

```
# Encode labels
label_encoder = LabelEncoder()
y_train = label_encoder.fit_transform(y_train)
y_test = label_encoder.transform(y_test)
```



MACHINE LEARNING MODELS

- Machine learning models are designed to learn patterns and relationships within data, making them well-suited for document classification. Here are the key concepts to understand:
 - Supervised Learning: Document classification is primarily a supervised learning task. It means the model learns from labelled examples where the text document and its associated category are provided during training.
 - Feature Vectors: Text data must be transformed into numerical feature vectors for input for machine learning models. This is typically achieved through techniques like Bag of Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), or word embeddings.
 - Classification Algorithms: Machine learning models predict the category of a document by mapping its feature vector to one of the predefined categories. Common classification algorithms include Naive Bayes, Support Vector Machines (SVM), and Random Forest.



TOP 4 ALGORITHMS FOR DOCUMENT CLASSIFICATION

- 1. Naive Bayes:
 - Pros: Simple, efficient, and effective for text classification. Works well even with small datasets.
 - Cons: Assumes independence between features (words), which may not always hold in text data.
- 2. Support Vector Machines (SVM):
 - Pros: Effective for both linear and non-linear classification. Good at handling high-dimensional data like text.
 - Cons: It can be computationally expensive for large datasets.



```
from sklearn.naive_bayes import MultinomialNB

# Example with Naive Bayes
model = MultinomialNB()
model.fit(X_train, y_train)

from sklearn.metrics import accuracy_score, classification_report

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy}")
print(f"Classification Report:\n{classification_rep}")
```



Accuracy: 0.7575

Classification Report:

	precision	recall	f1-score	support
0	0.78	0.72	0.75	199
1	0.74	0.80	0.77	201
accuracy			0.76	400
macro avg	0.76	0.76	0.76	400
weighted avg	0.76	0.76	0.76	400



TOP 4 ALGORITHMS FOR DOCUMENT CLASSIFICATION

- 3. Random Forest:
 - Pros: Ensemble of decision trees, providing good performance and the ability to capture complex patterns.
 - Cons: May overfit on small datasets if not carefully tuned.
- 4. Logistic Regression:
 - Pros: Simple and interpretable. Works well when the relationship between features and labels is approximately linear.
 - Cons: It may not capture complex, non-linear patterns as effectively as other models.



```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
from sklearn.metrics import accuracy_score, classification_report

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy}")
print(f"Classification Report:\n{classification_rep}")
```



Accuracy: 0.7525

Classification Report:

	precision	recall	f1-score	support
0	0.80	0.67	0.73	199
1	0.72	0.83	0.77	201
accuracy			0.75	400
macro avg	0.76	0.75	0.75	400
weighted avg	0.76	0.75	0.75	400



CHOOSING THE RIGHT MODEL

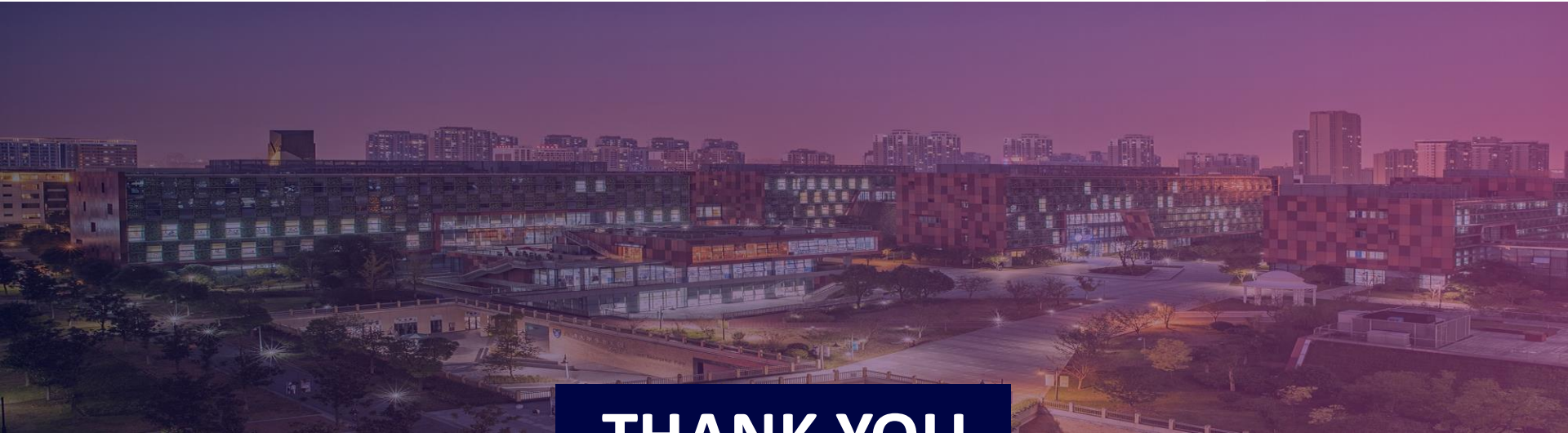
- Selecting the appropriate machine learning model for your document classification task depends on several factors:
 - Nature of the Data: Consider whether the relationship between features and labels is linear or non-linear. For complex patterns, ensemble models like Random Forest might be more suitable.
 - Size of the Dataset: Small datasets can benefit from simpler models like Naive Bayes, while larger datasets can handle more complex models like SVM or deep learning.
 - Computational Resources: Deep learning models can achieve state-of-the-art results but may require substantial computational resources and large amounts of data.
 - Interpretability: Depending on the application, the interpretability of the model may be a consideration. Naive Bayes and logistic regression models are more interpretable than deep learning models.
 - Experimentation: It's often beneficial to experiment with multiple models and assess their performance using appropriate evaluation metrics. Cross-validation can help identify the best-performing model for your specific task.
 - Ensemble Methods: Combining multiple models, such as using an ensemble of classifiers or stacking, can improve classification accuracy.



MODEL EVALUATION AND TUNING

- After selecting a model, the following steps involve training, evaluation, and potential hyperparameter tuning:
 - Training the Model: Train the selected machine learning model on the labelled dataset. Ensure you have a separate validation set for tuning hyperparameters.
 - Evaluation Metrics: Use appropriate evaluation metrics for document classification, such as accuracy, precision, recall, F1-score, and confusion matrices. Be cautious about overfitting to the evaluation metric.
 - Hyperparameter Tuning: Experiment with different settings to find the best model configuration. Techniques like grid search or random search can help with this process.
 - Cross-Validation: Implement cross-validation to ensure the model's performance is consistent and to mitigate the risk of overfitting.





THANK YOU



VISIT US

WWW.XJTLU.EDU.CN



FOLLOW US



Xi'an Jiaotong-Liverpool University
西交利物浦大学

